

2024 年非专业级别软件能力认证第一轮

(CSP-J) 入门级 C++语言模拟试题

考生注意事项:

试题纸共有 10 页, 答题纸共有 1 页, 满分 100 分。请在答题纸上作答, 写在试题纸上的一律无效。

不得使用任何电子设备(如计算器、手机、电子词典等)或查阅任何书籍资料。

一、单项选择题(共 15 题, 每题 2 分, 共计 30 分; 每题有且仅有一个正确选项)

1、在外部设备中, 绘图仪属于(B)

- A. 输入设备 **B. 输出设备** C. 辅(外)存储器 D. 主(内)存储器

2、RAM 中的信息是(B)。

- A. 生产厂家预先写人的
C. 防止计算机病毒侵入所使用的
- B. 计算机工作时随机写人的**
D. 专门用于计算机开机时自检用的

3、以下二进制数的值与十进制数 23.456 的值最接近的是(D)

- A. 10111.0101 B. 11011.1111 C. 11011.0111 **D. 10111.0111**

4、与十进制数 1770.625 对应的八进制数是(A)。

- A. 3352.5** B. 3350.5 C. 3352.1161 D. 3350.1151

5、64KB 的存储器用十六进制表示, 它的最大的地址码是(B)。

- A. 10000 **B. FFFF** C. 1FFFF D. EFFFF

6、设全集 $E = \{1, 2, 3, 4, 5\}$, 集合 $A = \{1, 4\}$, $B = \{1, 2, 5\}$, $C = \{2, 4\}$, 则集合 $(A \cap B) \cup \sim C$ 为(D)。

- A. $\{1\}$ B. $\{3, 5\}$ C. $\{1, 5\}$ **D. $\{1, 3, 5\}$**

7、字符串“ababacbab”和字符串“abcba”的最长公共子串是(B)

- A. abcba **B. cba** C. abc D. ab

8、线性表若采用链表存储结构, 要求内存中可用存储单元地址(D)

- A. 必须连续 B. 部分地址必须连续

C. 一定不连续

D. 连续不连续均可

9、设全集 $I=\{a, b, c, d, e, f, g\}$, 集合 $A=\{a, b, c\}$, $B=\{b, d, e\}$, $C=\{e, f, g\}$, 那么集合 $(A-B) \cup (\sim C \cap B)$ 为(A)

A. $\{a, b, c, d\}$

B. $\{a, b, d, e\}$

C. $\{b, d, e\}$

D. $\{b, c, d, e\}$

10、地面上有标号为 A、B、C 的 3 根细柱, 在 A 柱上放有 10 个直径相同中间有孔的圆盘, 从上到下依次编号为 1, 2, 3, ……., 将 A 柱上的部分盘子经过 B 柱移入 C 柱也可以在 B 柱上暂存。如果 B 柱上的操作记录为: “进, 进, 出, 进, 进, 出, 出, 进, 进, 出, 进, 出, 出”, 那么, 在 C 柱上, 从下到上的盘子的编号为(D)

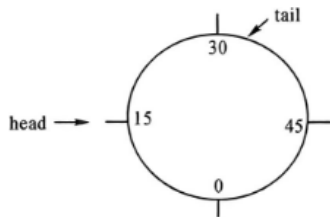
A. 2、4、3、6、5、7

B. 2、4、1、2、5、7

C. 2、4、3、1、7、6

D. 2、4、3、6、7、5

11、如图所示的循环队列中元素数目是(C)。其中 $\text{tail}=32$ 指向队尾元素, $\text{head}=15$ 指向队头元素的前一个空位置, 队列空间 $m=60$ 。



A. 42

B. 16

C. 17

D. 41

12、若用一个大小为 6 的数组来实现循环队列, 且当 rear 和 front 的值分别为 0 和 3。当从队列中删除一个元素, 再加入两个元素后, rear 和 front 的值分别为(B)

A. 1 和 5

B. 2 和 4

C. 4 和 2

D. 5 和 1

13. 一棵完全二叉树上有 1001 个结点, 其中叶子结点的个数是 (D)。

A. 250

B. 500

C. 254

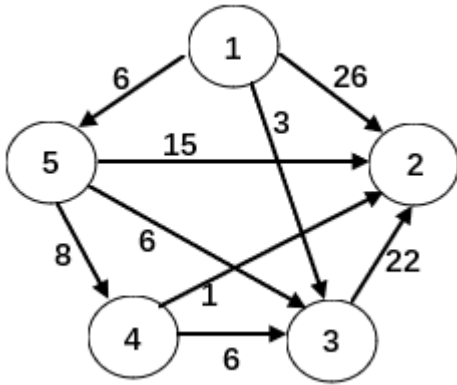
D. 501

解释: 法一: 设度为 0 结点 (叶子结点) 个数为 A, 度为 1 的结点个数为 B, 度为 2 的结点个数为 C, 有 $A=C+1$, $A+B+C=1001$, 可得 $2C+B=1000$, 由完全二叉树的性质可得 $B=0$ 或 1, 又因为 C 为整数, 所以 $B=0$, $C=500$, $A=501$, 即有 501 个叶子结点。

法二: $2^{10}-1=1023$, 故树高为 10, 最后一层少了 $1023-1001=22$ 的结点; 第十层结点最多: $2^{(10-1)}=512$, 最后的 22 个结点为空, 上一层为叶子节点, 22 个

孩子能产生 11 个父节点（叶子结点），即 $512-11=501$ 。

14. 使用 Dijkstra 算法求下图中从顶点 1 到其余各顶点的最短路径，将当前找到的从顶点 1 到顶点 2、3、4、5 的最短路径长度保存在数组 `dist` 中，求出第二条最短路径后，`dist` 中的内容更新为：(C)



- A. 26、3、14、6
- B. 25、3、14、6
- C. 21、3、14、6
- D. 15、3、14、6

15. 一个有 n 个顶点的简单有向图最多有 (B) 条边

- A. n
- B. $n(n-1)$
- C. $n(n-1)/2$
- D. $2n$

二、阅读程序（程序输入不超过数组或字符串定义范围；判断题正确填√，错误填×；出特殊说明外，判断题 2 分，选择题 3 分，合计 30 分）

(1)

```
01 #include<iostream>
02 void solve1(int n, int k) {
03     k ^= k >> 1;
04     while (~--n) std::cout << (k >> n & 1);
05     std::cout << std::endl;
06 }
```

```

07 void solve2(int n, int k, int flag) {
08     if (n == 0) {
09         return;
10     }
11     if (k < (1 << n - 1)) {
12         std::cout << flag;
13         solve2(n - 1, k, 0);
14     }
15     else {
16         std::cout << (flag ^ 1);
17         solve2(n - 1, k - (1 << n - 1), 1);
18     }
19 }
20 int main() {
21     int n, k;
22     std::cin >> n >> k;
23     solve1(n, k);
24     solve2(n, k, 0);
25     return 0;
26 }

```

判断题

1. 第二行输出的一定是长度为 n 的若干个 0 或 1 (✓)
2. 当输入 “5 1” 时，输出的第二行是 “00001” (✓)
3. 当输入 “10 20” 时，输出的第一行和第二行相等 (✓)

选择题

1. $\text{solve1}(n, k)$ 的时间复杂度为 (A)
A. n B. $n \log n$ C. n^2 D. 2^n
5. $\text{solve2}(n, k)$ 的时间复杂度为 (A)
A. n B. $n \log n$ C. n^2 D. 2^n
6. 当输入 “6 16”，输出的第一行为 (C)
A. 000000 B. 100100 C. 110110 D. 111111

Solve1 和 solve2 均为求 n 位的第 k 个格雷码

1. ✓ solve1 输出是 $k \gg n \& 1$ ，是 k 先右移 n 位再和 1 做与运算，结果只会是 0 和 1，循环次数是 $\sim n$ ，是循环 n 次（每次会 -1 再判断，判断 $\sim$ 是按位取反，只有全是 1 时会结束循环（即 -1 ）
2. ✓ 计算即可
3. ✓
4. A
5. A
6. C 可以用 solve2 计算比较方便。

(2)

```
1 #include <bits/stdc++.h>
2 using namespace std;

3 int main() {
4     string s;
5     cin >> s;
6     int length = s.size();
7     int len = (length + 1) / 2;
8     int i, j;
9     for (i = len - 1; i >= 0; i--) {
10         for (j = i + 1; j < length; j++) {
11             if (s[j] == s[j % (i + 1)]) {
12                 continue;
13             } else {
14                 break;
15             }
16         }
17         if (j == length && (j - 1) % (i + 1) == i) {
18             break;
19         }
20     }
21     if (i < 0) {
22         cout << false;
23     }
```

```
24     for (int t = 0; t < i + 1; t++) {
25         cout << s[t];
26     }
27     return 0;
28 }
```

判断题

1. 第九行 for 循环里的第一个表达式改为 `i=length`, 程序没有任何影响。(×)
2. 这个程序是给出一个非空的字符串, 判断这个字符串是否是由它的一个子串进行多次首尾拼接构成的。(√)
3. 输入字符串 "abcbca", 输出为 "abc"。(×)

选择题

4. 输入 "abcdefg", 则输出为 (C)。
A. abcdefg B. false C. 0 D. F
5. 输入 "123123123123", 则输出为 (C)。
A. 123 B. 123123 C. 123123123 D. 123123123123

解释: 程序是判断字符串是否是由它的一个子串进行多次首尾拼接构成的。
例如, "abcbcbcb" 满足条件, 因为它是由 "abc" 首尾拼接而成的, 而 "abcb" 则不满足条件。如果字符串满足上述条件, 则输出最长的满足条件的子串。
最长子串拼接, 故最长子串最多只可能有主串的一半长, 故第九行 `i` 不能等于 `length`。

三、完善程序 (单选题, 每小题 3 分、合计 30 分)

1. (进制转换) 输入一个十进制正整数 `num`, 把这个数转换为 `n` 进制并输出。

```
1  #include <stdio.h>
2  int s[100000];
3
4  int main() {
5      int num, n, i, j, sum = 0;
6      scanf("%d%d", &num, &n);
7      for (i = 0; ①; i++) {
8          sum = ②;
```

```
9         ③;  
10        s[i] = sum;  
11    }  
12    for (④; j >= 0; j--) {  
13        if (s[j] >= 10)  
14            printf("%c", ⑤);  
15        else  
16            printf("%d", s[j]);  
17    }  
18    printf("\n");  
19    return 0;  
20 }
```

1. ①处应填(A)

A. num!=0 B. n!=0 C. i<n D. i<num

2. ②处应填(C)

A. num/n
B. n/num
C. num%n
D. n%num

3. ③处应填(B)

A. sum=num/n
B. num=num/n
C. sum=n/num
D. num=n%num

4. ④处应填(A)

A. j=i-1
B. j=i
C. j=0
D. j=n

5. ⑤处应填(D)

A. s[j]+' A'

- B. `s[j]+' 0'`
- C. `s[j]-48`
- D. `s[j]+55`

2. (表达式求值) 输出一个包含数字和 '+'、'-'、'*'、'/' 的简单四则运算表达式，保证输入的表达式合法。试补全程序。

```
01 #include<iostream>
02 #include<stack>
03 using namespace std;
04 stack<int>num;
05 stack<char>op;
06
07 int check(char c) {
08     int a = 0;
09
10     if (c == '+' || c == '-')
11         a = 1;
12
13     if (c == '*' || c == '/')
14         a = ①;
15     return a;
16 }
17
18 void calc() {
19     int a, b;
20     char p;
21     b = num.top();
22     num.pop();
23     a = num.top();
24     num.pop();
25     p = op.top();
26     op.pop();
27
28     if (p == '+') {
29         num.push(a + b);
```



```

30     }
31
32     if (p == '*') {
33         num.push(a * b);
34     }
35
36     if (p == '-') {
37         num.push(a - b);
38     }
39
40     if (p == '/') {
41         num.push(a / b);
42     }
43 }
44
45 int main() {
46     string s;
47     cin >> s;
48     for (int i = 0; i < s.size(); i++) {
49         int data = 0;
50         if (s[i] >= '0' && s[i] <= '9') {
51             while (s[i] >= '0' && s[i] <= '9') {
52                 ②;
53                 i++;
54             }
55             num.push(data);
56             i--;
57         }
58         else {
59             while (③) {
60                 calc();
61             }
62             ④;
63         }
64     }

```

```
65     while (⑤)
66         calc();
67     cout << num.top();
68     return 0;
69 }
```

1. ①处应填(D)

A. -1 B. 0 C. 1 D. 2

2. ②处应填(D)

A. data = data + s[i]
B. data = data + (s[i] - '0')
C. data = data * 10 + s[i]
D. data = data * 10 + (s[i] - '0')

3. ③处应填(C)

A. check(s[i]) <= check(op.top())
B. check(s[i]) <= check(num.top())
C. op.size() > 0 && check(s[i]) <= check(op.top())
D. num.size() > 0 && check(s[i]) <= check(num.top())

4. ④处应填(A)

A. op.push(s[i])
B. num.push(s[i])
C. op.pop(s[i])
D. num.pop(s[i])

5. ⑤处应填(C)

A. op.empty()
B. num.empty()
C. !op.empty()
D. !num.empty()

用栈求，其中 op 是装运算符的，num 这个栈是装数字的

1. D 乘除的优先级要比加减高

- 2.D 输入的数字是十进制，而且要考虑从字符转数字
- 3.C op 是装运算符的栈，并且查看栈顶首先要栈不空
- 4.A
- 5.C 有运算符才能做运算

